



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Angebot

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 1. Mai 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Mohamed Sabri Jlizi	Fertig
1.1	Mohamed Sabri Jlizi	Fertig
1.2	Mohamed Sabri Jlizi	Fertig
2	Mohammed Alhalabi	Fertig
3	Lucas Neidahl	Fertig
3.1	Lucas Neidahl	Fertig
3.2	Lucas Neidahl	Fertig
4	Mohammad Khatib	Fertig
4.1	Mohammad Khatib	Fertig
4.2	Mohammad Khatib	Fertig
4.3	Mohammad Khatib	Fertig
5	Luan Hyseni	Fertig
5.1	Luan Hyseni	Fertig
5.2	Luan Hyseni	Fertig
5.3	Luan Hyseni	Fertig
6	Marvin Voigt	Fertig
6.1	Marvin Voigt	Fertig
6.2	Marvin Voigt	Fertig
6.3	Marvin Voigt	Fertig
7	Lucas Neidahl	Fertig

Inhaltsverzeichnis

1	Einleitung	5
1.1	Zielsetzung	5
1.2	Motivation	6
2	Formale Grundlagen	7
3	Projektablauf	8
3.1	Meilensteine	9
3.1.1	Dokument-bezogene Meilensteine	9
3.1.2	Software-bezogene Meilensteine	9
3.2	Geplanter Ablauf als Gant-Diagramme	10
3.2.1	Dokument Bezogenes Gant-Diagramm	10
3.2.2	Software Bezogenes Gant-Diagramm	11
4	Projektumfang	12
4.1	Lieferumfang	12
4.2	Kostenplan	12
4.3	Funktionaler Umfang	12
5	Entwicklungsrichtlinien	14
5.1	Konfigurationsmanagement	14
5.2	Design- und Programmierrichtlinien	14
5.3	Verwendete Software	14
6	Projektorganisation	16
6.1	Schnittstelle zum Auftraggeber	16
6.2	Schnittstelle zu anderen Projekten	16
6.3	Interne Kommunikation	16
7	Glossar	17
	Glossar	17

Abbildungsverzeichnis

3.1	Ablauf der Dokument-bezogenen Meilensteine	10
3.2	Ablauf der Software-bezogenen Meilensteine	11

1 Einleitung

Dieses Dokument stellt das Projektangebot für die Entwicklung der Softwarelösung SkyOrchestrator dar. Es definiert die Zielsetzung, Motivation und Rahmenbedingungen des Projekts und dient sowohl dem Auftraggeber als auch dem Projektteam als verbindliche Grundlage für die Projektumsetzung.

Gleichzeitig unterstützt das Dokument das Projektteam beim Einstieg in die Entwicklung, indem es die Struktur, Rollenverteilung und die wesentlichen Anforderungen dokumentiert. Das Projekt wird im Rahmen des Softwareentwicklungspraktikums im Sommersemester 2025 am Institut für Robotik und Prozessinformatik der Technischen Universität Braunschweig durchgeführt.

Projektübersicht

SkyOrchestrator ist eine Softwareanwendung zur Planung, Simulation und Visualisierung von Drohnenshows als moderne und umweltschonende Alternative zu Feuerwerken. Nutzerinnen und Nutzer sollen in die Lage versetzt werden, über eine intuitive Benutzeroberfläche individuelle Choreografien mit mehreren Drohnen zu entwerfen und in einer digitalen Umgebung zu simulieren.

1.1 Zielsetzung

Ziel des Projekts ist die Entwicklung einer modularen und benutzerfreundlichen Softwarelösung, mit der Nutzer:innen komplexe Drohnenshows konfigurieren und simulieren können. Die Anwendung soll die Erstellung von Bewegungsabläufen, Formationen und Lichtanimationen mehrerer Drohnen ermöglichen.

Die wichtigsten Funktionen umfassen:

- Definition und Steuerung von Flugbewegungen und Formationen mehrerer Drohnen
- Anpassung von Lichtfarbe, Lichtintensität und Timing einzelner Drohnen
- Festlegung der Anzahl und Startpositionierung der Drohnen
- Pfaddefinition über Koordinaten oder Vektoren im 3D-Raum

- Simulation der Choreografien in annähernder Echtzeit
- Zeitliche Steuerung und Ablaufplanung der gesamten Show

Optional geplante Erweiterungen:

- Kollisionsvermeidung durch algorithmische Analyse der Flugbahnen
- Optimierte Pfadberechnung zur Reduzierung von Flugzeit und Energieverbrauch
- Synchronisation der Choreografien mit Musik
- Export von Choreografiedaten (JSON/XML) für den Einsatz mit realen Drohnensystemen

1.2 Motivation

Das Projekt SkyOrchestrator wurde im Rahmen des SEP ausgewählt, um ein praxisnahes, innovatives und technisch anspruchsvolles Softwarevorhaben umzusetzen. Drohnenshows bieten eine attraktive, moderne Alternative zu traditionellen Feuerwerken – insbesondere im Hinblick auf Umweltfreundlichkeit, Sicherheit und kreative Ausdrucksmöglichkeiten.

Die Herausforderung liegt in der Entwicklung eines Systems, das sowohl kreative Gestaltung als auch technische Präzision vereint. Die Arbeit mit virtuellen Drohnen erlaubt es, reale Probleme der Robotik, Flugplanung und Visualisierung praxisnah zu simulieren und softwaretechnisch zu lösen.

2 Formale Grundlagen

Im Folgenden werden die formalen Grundlagen des Projekts sowie der genutzten Produkt- und Entwicklungsumgebungen beschrieben. Als Betriebssystem kommt Windows 11 zum Einsatz. Die Anwendung wird hauptsächlich mit Python programmiert; alternativ können auch C++ oder C verwendet werden. Die grafische Benutzeroberfläche (GUI) der entwickelten Software wird mit Tkinter vollständig in deutscher Sprache umgesetzt. Für Simulationen werden Programme verwendet, deren Benutzeroberflächen in deutscher Sprache verfügbar sind. Als Entwicklungsumgebungen kommen PyCharm¹ und Visual Studio Code² zum Einsatz. Zur Erstellung formaler Dokumente wird der Online-LaTeX-Editor Overleaf³ verwendet, der kollaboratives Arbeiten ermöglicht. Diagramme werden mit diagrams.net beziehungsweise Draw.io erstellt. Für die Versionskontrolle wird GitLab verwendet.

Fußnoten:

<https://www.jetbrains.com/pycharm/>

<https://code.visualstudio.com/>

<https://www.overleaf.com/>

<https://www.diagrams.net/>

<https://gitlab.com/>

3 Projektablauf

Das Projekt „SkyOrchestrator“ beginnt am 13. April 2025 mit einem Kick-off-Meeting, in dem Rollen, Verantwortlichkeiten sowie die technische Arbeitsumgebung besprochen werden. Im Anschluss wird im Zeitraum vom 14. bis zum 23. April das Projektangebot erarbeitet und eingereicht.

Parallel zur Ausarbeitung des Pflichtenhefts und der Abnahmespezifikation, die vom 24. April bis zum 14. Mai erstellt werden, wird ebenfalls das Architekturkonzept für die Software ausgearbeitet und die Basisklassen für die zentralen Softwareobjekte entwickelt.

Am 22. Mai 2025 findet die Zwischenpräsentation statt, bei der ein erster Prototyp vorgestellt wird. Dieser Prototyp wird im Zeitraum vom 15. bis zum 22. Mai entworfen und entwickelt.

Im Anschluss daran wird zwischen dem 23. Mai und dem 4. Juni der Fachentwurf erstellt, in dem die fachlichen Komponenten sowie deren Zusammenwirken beschrieben werden. Gleichzeitig beginnt ab dem 23. Mai die Entwicklung des Backends, wo die Flugroutenberechnung und die interne Logik der Drohnen erarbeitet wird. Die Backend-Entwicklung wird bis zum 25. Juni abgeschlossen.

Ab dem 11. Juni bis zum 25. Juni wird die Frontend-Entwicklung bearbeitet, in der die grafische Benutzeroberfläche für die Steuerung durch den Nutzer und Visualisierung der Drohnenshows erstellt wird.

Anschließend erfolgt eine Integrationsphase, in der Backend und Frontend zusammengeführt und getestet werden. Diese Integration soll spätestens am 30. Juni abgeschlossen sein, sodass ab dem 1. Juli eine Testphase beginnen kann. Parallel dazu werden die Testspezifikationen und Testprotokolle erstellt. Die Testphase inklusive Fehlerbehebung läuft bis zum 9. Juli 2025. Anschließend werden vom 10. bis zum 16. Juli letzte Optimierungen sowie die finale Softwareversion ausgearbeitet und dokumentiert.

Den Abschluss bildet die finale Abgabe und Präsentation der Projektergebnisse am 17. Juli 2025 im Rahmen des TDSE.

3.1 Meilensteine

3.1.1 Dokument-bezogene Meilensteine

Nummer	Meilenstein	Dokumente	Abgabetermin
1	Projektstart	-	13.04.2025
2	Angebot planen, erstellen, absenden	Angebot.pdf	23.04.2025
3	Pflichtenheft	Pflichtenheft.pdf	14.05.2025
4	Abnahmespezifikation	Abnahmespezifikation.pdf	14.05.2025
5	Zwischenpräsentation	Prototyp	22.05.2025
6	Fachentwurf	Fachentwurf.pdf	04.06.2025
7	Technischer Entwurf	TechnischerEntwurf.pdf	25.06.2025
8	Testspezifikation/Testprotokolle	Testdokumente, Tests	09.07.2025
9	TDSE/Abschlusspräsentation	Fertiges Projekt & Präsentation	17.07.2025

3.1.2 Software-bezogene Meilensteine

Nummer	Meilenstein	Werk	Abschlusstermin
1	Architekturkonzept erstellen	Architektur Übersicht	14.05.2025
2	Basisklassen entwickeln	Basisklassen (Code-Repository)	14.05.2025
3	Prototyp entwerfen	Prototyp, grober Entwurf	22.05.2025
4	Backend entwickeln	Backend-Code und Logik	25.06.2025
5	Frontend und GUI entwickeln	Frontend-Code, GUI	25.06.2025
6	Integration Backend und Frontend	funktionierender Prototyp (Testversion)	30.06.2025
7	Prototyp testen und Fehlerbehebung	Testprotokolle, Bugfixing	09.07.2025
8	Finalisierung der Software	Finale Version	17.07.2025

3.2 Geplanter Ablauf als Gant-Diagramme

3.2.1 Dokument Bezogendes Gant-Diagramm

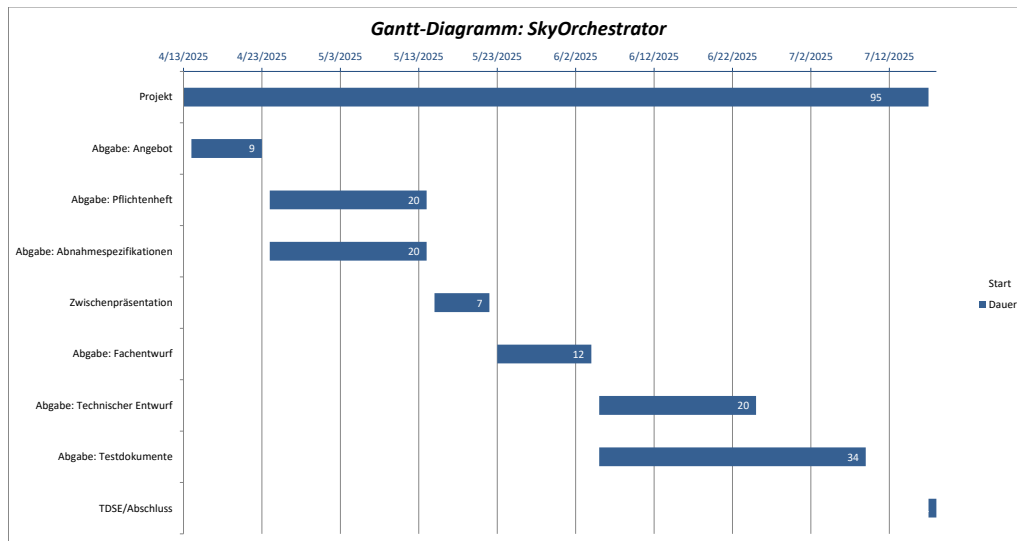


Abbildung 3.1: Ablauf der Dokument-bezogenen Meilensteine

3.2.2 Software Bezogendes Gant-Diagramm

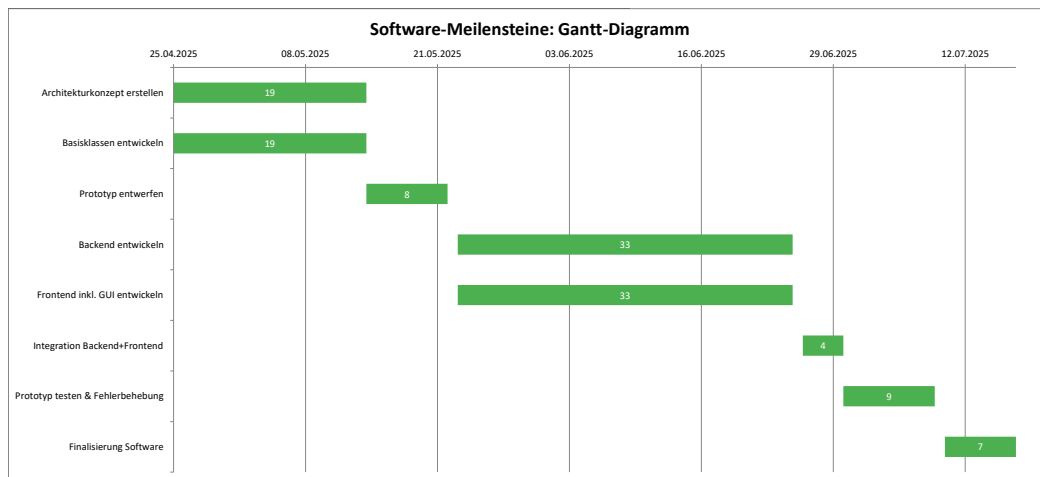


Abbildung 3.2: Ablauf der Software-bezogenen Meilensteine

4 Projektumfang

4.1 Lieferumfang

Am Ende des Projekts wird dem Kunden eine funktionsfähige Softwareanwendung übergeben, mit der sich Drohnenshows planen und simulieren lassen. Zum Lieferumfang gehören die ausführbare Anwendung selbst, der vollständige Quellcode sowie eine ausführliche technische Dokumentation. Außerdem werden sämtliche während des Projekts erstellten Dokumente übergeben, darunter das Angebot, das Pflichtenheft, der Fach- und Technische Entwurf sowie die Testdokumentation. Zusätzlich erhalten die Nutzer eine Bedienungsanleitung, um die Software intuitiv verwenden zu können. Optional soll eine Exportfunktion bereitgestellt werden, mit der sich die erstellten Shows in gängigen Formaten wie JSON oder XML speichern lassen, um sie später mit realen Drohnensystemen weiterzuverarbeiten. Die Software wird primär für Windows 11 entwickelt, ist jedoch auch mit Windows 10 sowie gängigen Linux-Distributionen kompatibel. Für die Nutzung wird ein Rechner mit aktueller Hardwareausstattung sowie einer installierten Python-Umgebung benötigt.

4.2 Kostenplan

Für das Projekt wird ein Gesamtaufwand von etwa 1.080 Arbeitsstunden angenommen. Diese Zahl ergibt sich aus einer Schätzung von rund 180 Stunden pro Teammitglied bei insgesamt sechs Teammitgliedern. Bei einem kalkulierten Stundensatz von 100 Euro pro Stunde ergeben sich daraus hypothetische Projektkosten in Höhe von 108.000 Euro. Diese Schätzung dient als Orientierung und verdeutlicht den Umfang und die Komplexität des Vorhabens.

4.3 Funktionaler Umfang

Die entwickelte Software mit dem Projektnamen *SkyOrchestrator* wird verschiedene zentrale Funktionen bereitstellen. Sie ermöglicht es Nutzer:innen, Drohnenshows individuell zu planen, zu gestalten und zu simulieren. Dabei können sie die Anzahl der eingesetzten Drohnen festlegen, Bewegungsbahnen durch Koordinaten oder Vektoren definieren und Formationen sowie Muster erstellen. Zusätzlich lassen sich Lichtfarbe, Lichtintensität, Bewegungsform und Geschwindigkeit der einzelnen Drohnen individuell konfigurieren. Auch die Dauer der Show und der zeitliche Ablauf können präzise gesteuert werden. Die Ergebnisse werden in einer grafischen Visualisierung dargestellt, die eine möglichst flüssige Darstellung nahe an Echtzeit ermöglicht, wobei aufgrund von begrenzter Rechenleistung leichte Verzögerungen nicht ausgeschlossen werden können.

Die Benutzeroberfläche (GUI) wird so gestaltet sein, dass Nutzer:innen intuitiv alle wichtigen Einstellungen vornehmen können. Über einfache Eingabefelder können Parameter wie die Anzahl der Drohnen, ihre Bewegungsprofile und die gewünschten Lichtanimationen definiert werden. Flugbahnen können direkt gezeichnet oder durch vordefinierte Muster erstellt werden, während eine Timeline-Ansicht eine gezielte Steuerung von Effekten und Formationswechseln innerhalb der Show ermöglicht. Zudem wird es möglich sein, die Show auf Musik zu synchronisieren, indem Audio-Dateien eingebunden werden.

Als optionale, erweiterte Funktionen sind unter anderem eine automatische Kollisionsvermeidung, eine optimierte Pfadplanung für jede Drohne, die Integration von Musik sowie die Möglichkeit, die geplanten Choreografien in Formaten wie JSON oder XML zu exportieren, vorgesehen. Diese Erweiterungen sollen die Software zu einem vielseitig einsetzbaren Werkzeug für die Planung realer Drohnenshows weiterentwickeln.

5 Entwicklungsrichtlinien

5.1 Konfigurationsmanagement

Wir verwenden ein zentrales GitHub-Repository zur Versionierung des Codes. Jedes Teammitglied hat Zugriff darauf. Die Ordnerstruktur ist klar unterteilt in `simulation/`, `models/`, `interface/`, `data/` usw. Es gilt die Regel, dass vor jedem Commit der Code ausführbar und fehlerfrei sein muss. Bei jedem Push muss ein Kommentar zur Änderung gegeben werden. Zusätzlich wird `.venv/` in der `.gitignore` ausgeschlossen, um virtuelle Umgebungen nicht mit ins Repository zu übernehmen. Zur Entwicklung neuer Funktionen und Fehlerbehebungen werden separate Branches verwendet. Änderungen werden erst nach erfolgreichem Testen auf den Hauptbranch gemerged. Die Commit-Kommentare erfolgen auf deutscher Sprache. Zur Aufgabenstrukturierung werden GitHub-Issues genutzt, größere Abschnitte des Projekts werden zusätzlich über Meilensteine organisiert. Nach Abschluss wichtiger Projektphasen werden Tags gesetzt, um den jeweiligen Versionsstand zu kennzeichnen.

5.2 Design- und Programmierrichtlinien

Wir orientieren uns an den PEP8-Styleguide für Python. Alle Teammitglieder achten auf einheitliche Einrückung, sprechende Variablennamen und sinnvolle Kommentare. Es werden für alle Funktionen Docstrings verwendet. Es können Tools wie `PyLint` oder `Black` zur Stilprüfung verwendet werden. Alle Variablennamen, Funktionen und Klassennamen werden in englischer Sprache benannt, um eine einheitliche Lesbarkeit zu gewährleisten.

5.3 Verwendete Software

PyBullet wird aktuell als Hauptsimulationsumgebung verwendet. Ein späterer Umstieg auf MuJoCo ist möglich, falls sich Anforderungen ergeben, die eine präzisere physikalische Simulation erfordern. Ob dieser Wechsel passieren wird, wird im weiteren Projektverlauf entschieden. Als IDE verwenden die meisten Teammitglieder PyCharm oder VS Code. Für die Versionskontrolle wird Git verwendet, mit einem zentralen Repository auf GitHub. Die JSON-Dateien zur Konfiguration der Shows werden manuell oder später über eine GUI erstellt. Diese enthalten Informationen wie Position, Farbe, Effekt und Startzeit. Alternativ können zeitbasierte Bewegungen als CSV-Format gespeichert werden, z.B für Flugpfade mit Zeitstempeln. Beide Formate sind mit realen Drohnensystemen grundsätzlich kompatibel und ermöglichen eine

spätere Weiterverarbeitung außerhalb der Simulation. Für grafische Benutzeroberflächen ist Tkinter vorgesehen. Falls 3D-Darstellungen komplexer werden, könnten später Unity oder Mujoco genutzt werden. Die Softwaredokumentation wird mit LaTeX auf der Plattform Overleaf erstellt.

6 Projektorganisation

6.1 Schnittstelle zum Auftraggeber

Der Auftraggeber bzw. der Betreuer kann über den TU-Messenger sowie über die TU-Mail erreicht werden. Über diese Kommunikationswege können Fragen und Probleme zum Projekt besprochen und geklärt werden. Fünf Team-Meetings sind eingeplant, um die Fortschritte und Entwicklungen des Projektes regelmäßig zu besprechen.

Zusätzlich besteht die Möglichkeit, Sonntag vor den Abgabeterminen dem Betreuer die vorläufigen Dokumente zu teilen. Dadurch können wir ein erstes Feedback einholen und optional die Dokumente vor den Abgaben überarbeiten.

6.2 Schnittstelle zu anderen Projekten

Das Projekt SkyOrchestrator ist auf eine Gruppe ausgelegt und hat keine weiteren Schnittstellen zu anderen Projekten.

6.3 Interne Kommunikation

Zum Start des Projekts wurden im Team ein Scrum Master und ein Teamleiter gewählt. Der Scrum Master sorgt für die Einhaltung der Scrum-Methodik, moderiert Meetings und unterstützt das Team bei der Selbstorganisation. Der Teamleiter übernimmt organisatorische Aufgaben und koordiniert die Kommunikation innerhalb des Teams.

Der Hauptkommunikationsweg des Teams ist der TU-Messenger. Hierüber teilen wir die Aufgaben untereinander auf und lösen Probleme. Falls nötig, organisiert der Teamleiter über diesen Weg zusätzliche Treffen.

Das Team arbeitet außerdem mit Overleaf, um gemeinsam die Dokumente zu erstellen und bietet dadurch eine weitere Ebene der Kommunikation. Durch die Kommentarfunktion sowie den eingebetteten Chat auf der Seite können wir die Dokumente gemeinsam bearbeiten. Des Weiteren sind wöchentlich Treffen geplant, um die Aufgaben umfangreicher face-to-face zu besprechen.

Zunächst arbeitet das Team gemeinsam an den anfallenden Aufgaben und ist nicht in kleinere Gruppen aufgeteilt. Sollte sich im Laufe des Projektes herausstellen, dass eine Spezialisierung sinnvoll ist, kann sich das Team dynamisch entsprechend seiner individuellen Stärken in kleinere Gruppen unterteilen. Diese offene Struktur fördert die Zusammenarbeit und ermöglicht es allen, sich aktiv einzubringen und voneinander zu lernen.

7 Glossar

Abnahmespezifikation Dokumentation der Kriterien, anhand derer das Projekt am Ende formell abgenommen wird.

Angebot Schriftliches Dokument, das dem Auftraggeber geplante Leistungen, Zeitplan und Kosten eines Projekts unterbreitet.

Architekturkonzept Es beschreibt die grobe Struktur der Software, also welche Hauptkomponenten es gibt, wie sie miteinander funktionieren und welche Technologien dafür verwendet werden.

Backend Softwarekomponente, die die interne Logik und Berechnungen, wie z. B. die Flugroutenberechnung für Drohnenshows, übernimmt.

Basisklassen Basisklassen sind erste fertige Programmierklassen, die die wichtigsten Objekte des Systems darstellen. Sie enthalten noch nicht die ganze Logik, sondern definieren die Grundstruktur.

Black Tool zur automatischen Formatierung von Python-Code gemäß PEP8, um einheitlichen Stil im Team sicherzustellen.

Bugfixing Prozess der Identifikation und Behebung von Fehlern in der Software während der Testphase.

CSV Einfaches Tabellenformat zur Speicherung von zeitbasierten Daten, z. B. Flugpfade über eine definierte Zeitachse hinweg.

Docstring Mehrzeiliger Kommentar direkt unter einer Funktion oder Klasse, der Zweck und Verwendung beschreibt.

Fachentwurf Ausarbeitung der fachlichen Systemstruktur, insbesondere welche Funktionen das System erfüllt und wie diese logisch zusammenhängen.

Finale Version Die endgültige, abgeschlossene Version der Software, die alle Anforderungen erfüllt und für die Abgabe bereit ist.

Flugroutenberechnung Berechnung der Trajektorien und Bewegungen von Drohnen für die Show, eine zentrale Funktion des Backends.

Frontend Grafische Benutzeroberfläche und Visualisierungskomponente, die es Nutzern ermöglicht, die Drohnenshow zu steuern und darzustellen.

Integrationsphase Projektphase, in der Backend und Frontend zusammengeführt und getestet werden, um einen funktionsfähigen Prototypen zu erstellen.

JSON Textbasiertes Datenformat zur strukturierten Speicherung von Daten. Im Projekt werden Show-Konfigurationen (Positionen, Farben, Effekte) in JSON gespeichert.

Kick-off-Meeting Erstes offizielles Treffen aller Projektbeteiligten zur Planung und Rollenverteilung.

MuJoCo Präzise Simulationsumgebung, die physikalisch realistische Bewegungsabläufe ermöglicht.

PEP8 Offizieller Styleguide für Python, der Konventionen für sauberen, gut lesbaren Code festlegt.

Pflichtenheft Dokument beschreibt, wie und womit die Anforderungen des Auftraggebers technisch umgesetzt werden sollen.

Projektstart Eröffnungsphase des Projekts, meist durch ein Kick-off-Meeting eingeleitet, in dem Rollen und Verantwortlichkeiten geklärt werden.

PyBullet Physik-Engine zur Simulation von Bewegungen in Robotik- und Drohnenszenarien. Wird in diesem Projekt zur Darstellung der Drohnenshow verwendet.

SkyOrchestrator Name des Projekts, das eine Software zur Planung, Simulation und Steuerung von Drohnenshows entwickelt.

TDSE „Tag der jungen Softwareentwickler:innen“ – die Abschlusspräsentation des Projekts.

Technischer Entwurf Technische Detaillierung des Systems mit konkreten Softwarekomponenten, Schnittstellen und Technologien.

Testphase Projektphase, in der die Software getestet wird, um Fehler zu finden und die Funktionalität zu validieren.

Testprotokoll Schriftliche Aufzeichnung der Testergebnisse, inklusive dokumentierter Fehler oder Abweichungen.

Testspezifikation Dokument, das definiert, welche Tests zur Sicherstellung der Systemqualität geplant sind.

Tkinter Integrierte Bibliothek in Python zur Erstellung von grafischen Benutzeroberflächen.

URDF „Unified Robot Description Format“ – ein XML-basiertes Dateiformat zur Beschreibung von Roboter- und Drohnenmodellen.

Zwischenpräsentation Präsentation eines frühen Prototyps, um den Fortschritt des Projekts zu zeigen und Feedback zu sammeln.